Context-Aware Extension to GraDeT-HTR

Dual-Path Training with Margin Loss for
Bengali Handwritten Text Recognition

Technical Report — Extension Work Documentation

Md. Mahmudul Hasan Ahmed Nesar Tahsin Choudhury Mahmudul Hasan
Md. Mosaddek Khan

Computer Science and Engineering, University of Dhaka

February 24, 2026

Abstract

This report documents the context-aware extension to GraDeT-HTR, a Bengali Handwritten Text Recognition system published at EMNLP 2025 System Demonstrations. The base system employs a decoder-only transformer (DTrOCR) with grapheme-based tokenization, achieving state-of-the-art performance on Bengali HTR benchmarks (CER 6.19%, WER 14.20% on BN-HTRd). We extend the architecture with a *dual-path training* paradigm that leverages the previously-recognized word as linguistic context. A novel loss function combines standard cross-entropy for both contextual and isolation paths with a margin penalty term $\lambda \cdot \max(0, \mathcal{L}_{\text{ctx}} - \mathcal{L}_{\text{iso}})$ that penalizes samples where context *hurts* recognition accuracy. The extension modifies 6 source files, introduces a new `ContextAwareDataset` class, implements fixed-length context padding for batch-compatible training, and includes a comprehensive diagnostic test suite of 18 test cases. We detail the theoretical foundation, architecture design with embedding reordering, the complete data flow from context to output, and provide side-by-side code comparisons marking all new additions.

Contents

Chapter 1

Introduction and Motivation

1.1 The Base System — GraDeT-HTR (EMNLP 2025)

GraDeT-HTR [**?**] is a **Gra**pheme-aware **De**coder-only **T**ransformer for Bengali **H**andwritten **T**ext **R**ecognition, published at the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP), System Demonstrations track. The system establishes a three-stage pipeline:

1. **Line Segmentation**: BN-DRISHTI (YOLOv5) detects text lines from full-page images, applying Hough line and affine transformations for deskewing.
2. **Word Segmentation**: A second YOLOv5 model segments each line into individual word images.
3. **Text Recognition**: DTrOCR, a decoder-only transformer, converts each 32×128 word image into Bengali grapheme sequences using autoregressive decoding.

The recognition model (DTrOCR) is the core architectural contribution:

- **Visual Encoding**: ViT patch embeddings divide each 32×128 image into 4×8 non-overlapping patches, producing $N = (32/4) \times (128/8) = 128$ patch tokens, each linearly projected to 768 dimensions.
- **Text Decoding**: 12 GPT-2 transformer blocks with causal (autoregressive) self-attention process the concatenated sequence $[\text{IMAGE}_1, \ldots, \text{IMAGE}_{128}, \text{BOS}, t_1, \ldots, t_T, \text{EOS}]$.
- **Tokenization**: A custom Bengali grapheme tokenizer (BnGraphemizer) with 1296 graphemes (vocabulary size 1420 including special tokens) replaces standard BPE, providing one-to-one mapping between visually coherent units and model tokens.
- **Language Model Head**: A linear projection $\mathbb{R}^{768} \to \mathbb{R}^{1420}$ maps hidden states to vocabulary logits.

The model achieves state-of-the-art results with only 87M parameters, trained from scratch without a pretrained Bengali language model:

1.2 Motivation for Context-Aware Recognition

The base DTrOCR system processes each word image *in complete isolation*—the model has no knowledge of what preceded or follows the current word. In handwritten documents, however, adjacent words carry strong linguistic context:

Table 1.1: GraDeT-HTR base system performance (EMNLP 2025).

Model	**BN-HTRd**		**Bongabdo**	
	CER (%)	WER (%)	CER (%)	WER (%)
Word_BnG_Random (Ours)	**6.19**	**14.20**	**8.68**	**23.56**
Word_BPE_Pretrained	7.57	16.30	11.61	26.99
Word_BPE_Random	6.68	15.05	10.09	25.39
Gemini 2.5 Flash	19.39	32.49	37.42	56.39

- **Linguistic constraints**: Knowing that the previous word is a preposition constrains the space of plausible next words to nouns, pronouns, or adjectives.
- **Disambiguation**: Many Bengali grapheme sequences are visually similar. Context from the previous word can disambiguate between candidates that look identical in isolation.
- **Error recovery**: If a grapheme is partially obscured or poorly written, context provides a prior that can compensate for missing visual information.

This observation motivates the central question of this extension: *Can we inject linguistic context from the previously-recognized word into the decoder-only transformer to improve recognition accuracy?*

1.3 Contributions Summary

This extension introduces:

1. A **dual-path training paradigm** where each word image is processed twice per training step—once with the previous word as context and once in isolation—forcing the model to learn both context-aware and context-free recognition.
2. A **novel loss function** with a margin penalty term $\lambda \cdot \max(0, \mathcal{L}_{\text{ctx}} - \mathcal{L}_{\text{iso}})$ that explicitly penalizes samples where context *degrades* performance.
3. An **embedding reordering mechanism** that inserts context tokens before image patches in the transformer's input sequence, enabling causal attention from image patches to context tokens.
4. A **comprehensive diagnostic test suite** (18 test cases, 1020+ lines) validating dimension correctness, gradient flow, loss formula accuracy, and backward compatibility.

Chapter 2

Theoretical Foundation

2.1 Problem Formulation

Standard HTR. Given a word image $\mathbf{x} \in \mathbb{R}^{3\times 32\times 128}$, predict a grapheme sequence $\mathbf{y} = (y_1, y_2, \ldots, y_T)$ where each $y_t \in \mathcal{V}$ (vocabulary of 1420 tokens). The model factorizes the joint probability autoregressively:

$$P(\mathbf{y} \mid \mathbf{x}) = \prod_{t=1}^{T} P(y_t \mid y_{<t}, \mathbf{x}) \tag{2.1}$$

Context-Aware HTR. Given word image $\mathbf{x}$ *and* the previous word's text $\mathbf{c} = (c_1, c_2, \ldots, c_K)$, the contextual probability is:

$$P(\mathbf{y} \mid \mathbf{x}, \mathbf{c}) = \prod_{t=1}^{T} P(y_t \mid y_{<t}, \mathbf{x}, \mathbf{c}) \tag{2.2}$$

We define two per-sample negative log-likelihoods:

$$\mathcal{L}_{\text{iso}} = -\frac{1}{T-1} \sum_{t=1}^{T-1} \log P(y_t \mid y_{<t}, \mathbf{x}) \quad \text{(isolation path)} \tag{2.3}$$

$$\mathcal{L}_{\text{ctx}} = -\frac{1}{T-1} \sum_{t=1}^{T-1} \log P(y_t \mid y_{<t}, \mathbf{x}, \mathbf{c}) \quad \text{(contextual path)} \tag{2.4}$$

2.2 The Dual-Path Loss Function

Dual-Path Loss with Margin Penalty

For each sample i in a batch of size B:

$$\boxed{\mathcal{L} = \frac{1}{B} \sum_{i=1}^{B} \left[\underbrace{\mathcal{L}_{\text{ctx}}^{(i)}}_{\text{Term 1}} + \underbrace{\mathcal{L}_{\text{iso}}^{(i)}}_{\text{Term 2}} + \underbrace{\lambda \cdot \max\left(0,\ \mathcal{L}_{\text{ctx}}^{(i)} - \mathcal{L}_{\text{iso}}^{(i)}\right)}_{\text{Term 3: Margin Penalty}} \right]} \tag{2.5}$$

Term 1: Contextual Cross-Entropy ($\mathcal{L}_{\text{ctx}}$). Trains the model to recognize Bengali graphemes given both the word image *and* the previous word's text as context. This is the standard negative log-likelihood computed from the forward pass with context tokens prepended.

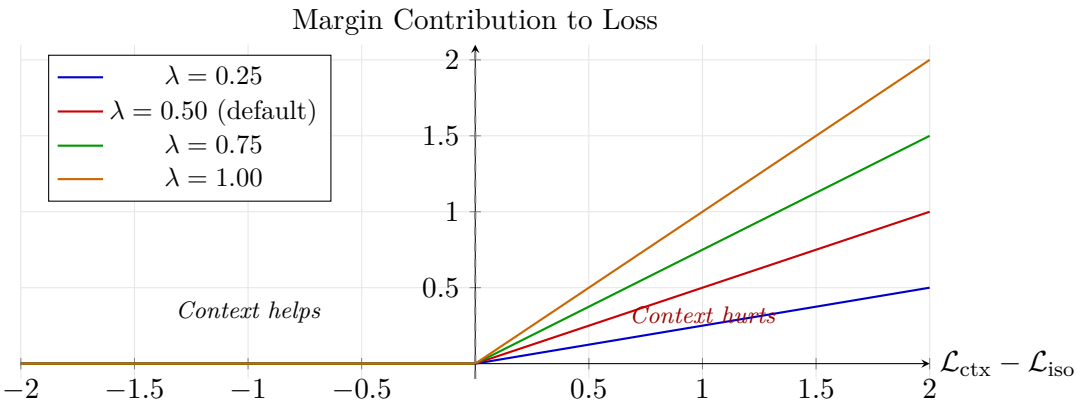

Figure 2.1: Margin penalty contribution as a function of the loss difference $\mathcal{L}_{\text{ctx}} - \mathcal{L}_{\text{iso}}$. The penalty activates only in the positive region (context is worse than isolation) and scales linearly with λ.

Term 2: Isolation Cross-Entropy ($\mathcal{L}_{\text{iso}}$). Trains the model to recognize the same word using only the image, without any context. This ensures the model maintains strong image-only recognition capability and does not become dependent on context.

Term 3: Margin Penalty ($\lambda \cdot \max(0, \mathcal{L}_{\text{ctx}} - \mathcal{L}_{\text{iso}})$). A hinge-like regularization term that activates *only when context hurts*—i.e., when the contextual path produces a higher loss (worse recognition) than the isolation path. When active, it provides an additional gradient signal pushing the model to ensure that context *always helps*.

Probability-to-loss-space conversion. The original formula is expressed in probability space as $\max(0, P_i - P_c)$, where $P_i = \exp(-\mathcal{L}_{\text{ctx}})$ is the contextual probability and $P_c = \exp(-\mathcal{L}_{\text{iso}})$ is the isolation probability. This fires when $P_i < P_c$, which is equivalent to $\mathcal{L}_{\text{ctx}} > \mathcal{L}_{\text{iso}}$ in loss space. We use the loss-space approximation $\max(0, \mathcal{L}_{\text{ctx}} - \mathcal{L}_{\text{iso}})$ which has the same activation condition and cleaner gradients.

2.3 Gradient Analysis

Let θ denote the model parameters (shared between both paths).

When margin is inactive ($\mathcal{L}_{\text{ctx}} \leq \mathcal{L}_{\text{iso}}$, context helps):

$$\frac{\partial \mathcal{L}}{\partial \theta} = \frac{\partial \mathcal{L}_{\text{ctx}}}{\partial \theta} + \frac{\partial \mathcal{L}_{\text{iso}}}{\partial \theta} \tag{2.6}$$

Both paths receive equal gradient signal. Training reduces both losses symmetrically.

When margin is active ($\mathcal{L}_{\text{ctx}} > \mathcal{L}_{\text{iso}}$, context hurts):

$$\frac{\partial \mathcal{L}}{\partial \theta} = (1 + \lambda) \cdot \frac{\partial \mathcal{L}_{\text{ctx}}}{\partial \theta} + (1 - \lambda) \cdot \frac{\partial \mathcal{L}_{\text{iso}}}{\partial \theta} \tag{2.7}$$

For $\lambda = 0.5$: the contextual gradient is *amplified* by factor $1.5\times$ while the isolation gradient is *reduced* by factor $0.5\times$. This is intuitive: when context is not helping, focus more gradient energy on improving the contextual path (which is underperforming) and less on the isolation path (which is already doing well).

For $\lambda = 1.0$: the isolation gradient is *zeroed out* and the contextual gradient is *doubled*, making the model entirely focus on improving context utilization for that sample.

Table 2.1: Comparison: original difficulty weighting vs. corrected margin term.

Aspect	**Difficulty Weighting (Original)**	**Margin Term (Corrected)**
Formula	$\bar{\mathcal{L}} = \text{mean}(w_i \cdot (\ell_i^{\text{ctx}} + \ell_i^{\text{iso}}))$ where $w_i \propto \ell_i^{\text{iso}}$	$\bar{\mathcal{L}} = \text{mean}(\ell_i^{\text{ctx}} + \ell_i^{\text{iso}} + \lambda \max(0, \ell_i^{\text{ctx}} - \ell_i^{\text{iso}}))$
Effect	Upweights samples where *isolation is hard*	Penalizes samples where *context is worse*
Asymmetry	No—both paths weighted equally per sample	Yes—contextual path gets stronger gradient when it underperforms
Goal	Focus on hard examples	Ensure context always helps

Recommended λ range: 0.3–1.0. We use $\lambda = 0.5$ as the default.

2.4 Per-Sample vs. Per-Batch Margin

The margin term operates *per-sample*, not per-batch. This is critical because within a single batch:

- Some samples may have informative context (the previous word is related to the current one)—for these, $\mathcal{L}_{\text{ctx}} < \mathcal{L}_{\text{iso}}$ and the margin is zero.
- Other samples may have irrelevant or misleading context (e.g., the first word of a new sentence)—for these, $\mathcal{L}_{\text{ctx}} > \mathcal{L}_{\text{iso}}$ and the margin activates.

A per-batch margin would average these effects, potentially missing the signal entirely. Per-sample computation ensures the penalty targets exactly the samples that need it.

The masked per-sample loss is computed as:

$$\ell_i = \frac{\sum_{t=1}^{S} m_t \cdot \ell_{i,t}}{\sum_{t=1}^{S} m_t} \tag{2.8}$$

where m_t is the attention mask (0 for padding, 1 for real tokens) and $\ell_{i,t}$ is the per-token cross-entropy loss for sample i at position t.

Chapter 3

Architecture Design

3.1 Base Architecture Recap

In the base DTrOCR (without context), the input sequence to the transformer is:

$$\mathbf{S}_{\text{base}} = [\underbrace{\text{IMG}_1, \ldots, \text{IMG}_{128}}_{\text{128 patch tokens}}, \underbrace{\text{BOS}, t_1, \ldots, t_N, \text{EOS}, \text{PAD}, \ldots}_{\text{128 target tokens (padded)}}] \tag{3.1}$$

Total sequence length: $128 + 128 = 256$ positions, matching `max_position_embeddings`.

3.2 Context-Aware Sequence Design

The extension prepends context tokens (from the previous word) *before* the image patches, creating the contextual sequence:

$$\mathbf{S}_{\text{ctx}} = [\underbrace{c_1, \ldots, c_K, \text{SEP}, \text{PAD}, \ldots}_{C \text{ context tokens (padded)}}, \underbrace{\text{IMG}_1, \ldots, \text{IMG}_{128}}_{\text{128 patches}}, \underbrace{\text{BOS}, t_1, \ldots, t_N, \text{EOS}, \text{PAD}, \ldots}_{\text{128 target tokens}}] \tag{3.2}$$

Total: $C + 128 + 128$ positions (where $C = 20$ by default).

Why prepend before image? Under causal attention, each position can only attend to positions at or before it:

- **Context tokens** (positions 0 to $C-1$): attend only to other context tokens—they serve as a "prefix prompt."
- **Image patches** (positions C to $C+127$): attend to context tokens *and* other image patches—the visual representation is *conditioned on context*.
- **Target tokens** (positions $C+128$ onwards): attend to everything—context, image, and previously generated tokens.

3.3 Embedding Reordering Mechanism

A key design challenge is that the **processor** outputs tokens in the order $[\text{context}, \text{target}]$ (since it concatenates token IDs), but the **model** needs them in the order $[\text{context}, \text{IMAGE}, \text{target}]$ (since image patches must be between context and target for correct causal attention). The model's `DTrOCRModel.forward()` performs this reordering:

1. Compute `token_embeddings` = Embedding(`input_ids`) — shape $[B, C{+}T, 768]$

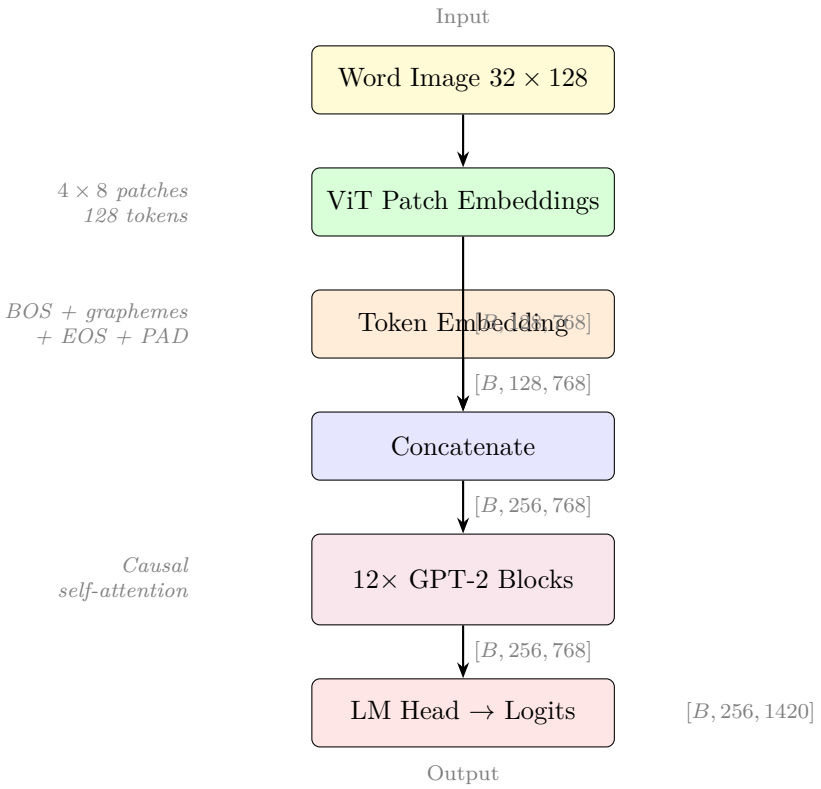

Figure 3.1: Base DTrOCR architecture (without context). The 32×128 word image is divided into 128 patch tokens, concatenated with target grapheme tokens, and processed by 12 GPT-2 blocks with causal attention.

2. Split: `context_emb` = `token_emb`$[:, : C, :]$ and `target_emb` = `token_emb`$[:, C :, :]$
3. Compute `patch_emb` = ViTPatchEmbeddings(`pixel_values`) — shape $[B, 128, 768]$
4. Concatenate: [`context_emb`, `patch_emb`, `target_emb`] — shape $[B, C{+}128{+}T, 768]$
5. Add positional embeddings covering the full reordered sequence

3.4 Causal Attention Pattern

3.5 Fixed-Length Context Padding

A critical implementation requirement is that all samples in a batch must have identical tensor shapes for `torch.stack()` in the DataLoader. Since different words have different numbers of context graphemes, naïve tokenization produces variable-length tensors—causing a runtime crash.

Solution. Pad all context tokens to a fixed `max_context_length` = 20:

1. Tokenize the previous word text → variable-length token sequence $(c_1, \ldots, c_k)$
2. Truncate to at most `max_context_length` − 1 = 19 tokens
3. Append SEP token (reusing `eos_token_id`)
4. Pad with `pad_token_id` to exactly 20 tokens

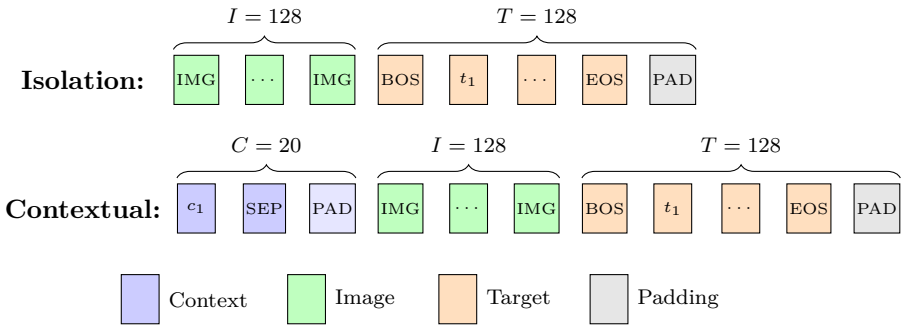

Figure 3.2: Sequence structure comparison between isolation mode (top) and contextual mode (bottom). Context tokens (C=20) are prepended before image patches. Total contextual sequence: $20 + 128 + 128 = 276$ positions.

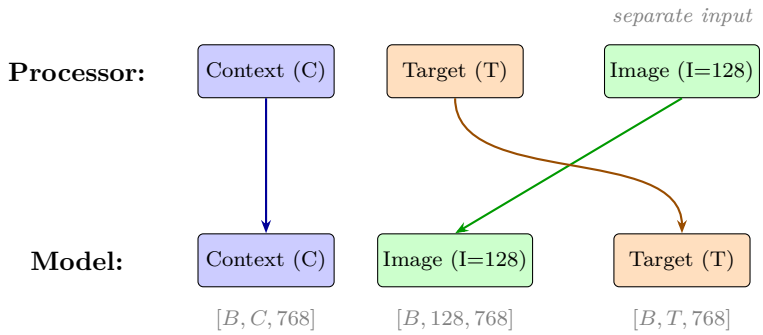

Figure 3.3: Embedding reordering: the processor outputs `[context, target]` token IDs, while the model reorders embeddings to `[context, IMAGE, target]` before passing through GPT-2 blocks.

5. Set attention mask: 1 for real tokens + SEP, 0 for padding

   Result: `context_length` = 20 for *every* sample, enabling uniform batching.

3.6 Position Embedding Budget

With context, the total sequence length becomes $C + I + T = 20 + 128 + 128 = 276$, exceeding the original `max_position_embeddings` = 256. The solution is to increase `max_position_embeddings` to 320, creating a new positional embedding matrix $\mathbf{P} \in \mathbb{R}^{320\times768}$. Positions 0–255 can be initialized from the pretrained model; positions 256–319 are randomly initialized and learned during context-aware fine-tuning. This adds approximately 49K parameters ($< 0.06\%$ of the 87M total), a negligible overhead.

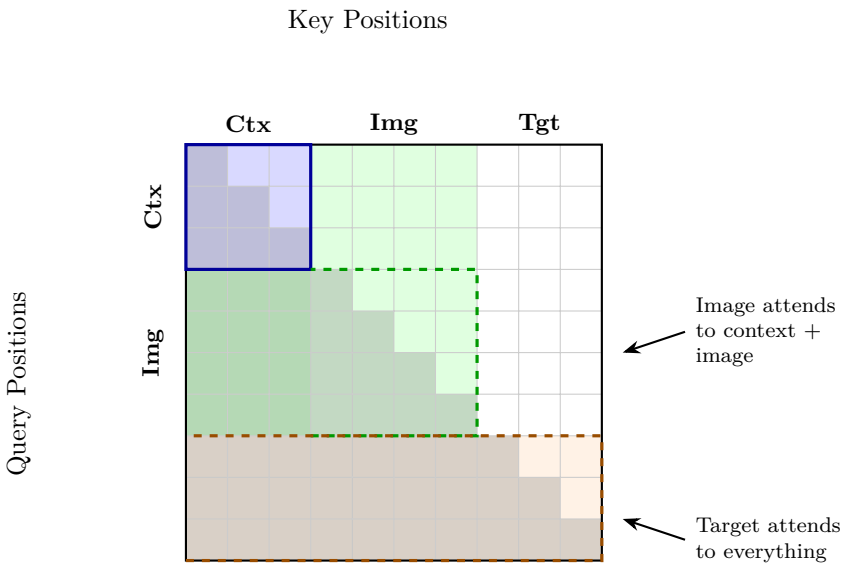

Figure 3.4: Causal attention pattern in context mode. Shaded cells indicate attended positions. Context tokens attend only to themselves; image patches attend to context and earlier image patches; target tokens attend to the full prefix (context + image + prior targets).

Chapter 4

Data Pipeline

4.1 JSON Data Format and Context Construction

Context-aware training uses JSON files (one per word image) containing:

```
{"output_path": "path/to/word_image.jpg", "text": "target_text"}
```

The `ContextAwareDataset` constructs previous-word context via a simple shift operation on the DataFrame:

```
df['prev_text'] = df['text'].shift(1).fillna("")
```

The first word of each document receives an empty string as context, making it equivalent to isolation mode.

4.2 ContextAwareDataset Class

Each call to `__getitem__` produces dual-path data by processing the same image twice:

Table 4.1: Dataset output dictionary structure per sample.

Key	**Type**	**Shape**	**Description**
`pixel_values`	FloatTensor	$[3, 32, 128]$	Preprocessed word image (shared)
`input_ids`	LongTensor	$[C{+}T]$	Context + target token IDs
`attention_mask`	LongTensor	$[C{+}T]$	1 for real, 0 for padding
`labels`	LongTensor	$[C{+}T]$	Same as input_ids (teacher forcing)
`context_length`	int	scalar	Always $= C = 20$
`input_ids_isolated`	LongTensor	$[T]$	Target-only token IDs
`attention_mask_isolated`	LongTensor	$[T]$	Target-only mask
`labels_isolated`	LongTensor	$[T]$	Target-only labels
`context_length_isolated`	int	scalar	Always $= 0$

4.3 Processor Context Tokenization Pipeline

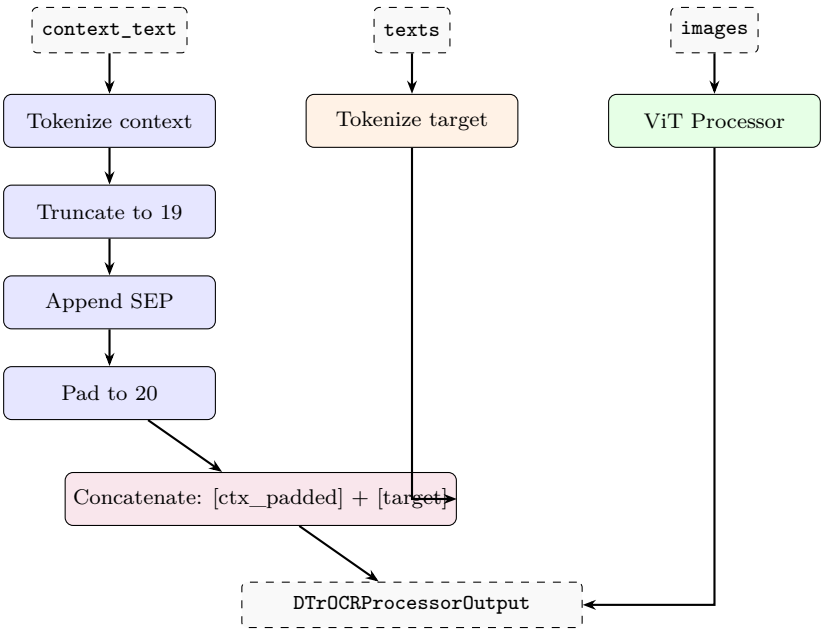

Figure 4.1: Processor data flow for context-aware mode. Context text is tokenized, truncated, given a SEP separator, and padded to a fixed length. It is then concatenated with target token IDs. The image is processed independently through the ViT processor.

Chapter 5

Training Phase

5.1 Training Loop Overview

The training script accepts configurable hyperparameters via `argparse`:

```
python train.py --context --margin_lambda 0.5 --batch_size 32 --epochs 10 --lr 1e-4
```

Mode selection: `-context` enables dual-path training with the `ContextAwareDataset`; without it, the standard single-path training runs with `HandwrittenDataset`.

5.2 Dual-Path Training Algorithm

5.3 Dimension Trace — Contextual Path

?? provides a complete tensor shape trace through the contextual forward pass for a concrete example with batch size B=4, context length C=20, target length T=128, image patches I=128, hidden dim H=768, vocabulary V=1420.

Table 5.1: Dimension trace through the contextual forward pass.

Stage	**Tensor**	**Shape**	**Notes**
Input	`pixel_values`	$[4, 3, 32, 128]$	RGB images
Input	`input_ids`	$[4, 148]$	C+T = 20+128
Input	`attention_mask`	$[4, 148]$	
Embed	`patch_embeddings`	$[4, 128, 768]$	ViT patches
Embed	`token_embeddings`	$[4, 148, 768]$	All tokens
Split	`context_embeddings`	$[4, 20, 768]$	First C=20
Split	`target_embeddings`	$[4, 128, 768]$	Remaining T=128
Reorder	`combined_embeddings`	$[4, 276, 768]$	C+I+T
PosMask	`position_ids`	$[1, 276]$	$0 \ldots 275$
PosMask	`combined_mask`	$[4, 276]$	Reordered
GPT-2	`hidden_states`	$[4, 276, 768]$	After 12 blocks
LM Head	`logits`	$[4, 276, 1420]$	Vocab scores
Loss	`skip_positions`	148	C+I = 20+128
Loss	`shift_logits`	$[4, 127, 1420]$	`logits[..., 148:-1, :]`
Loss	`shift_labels`	$[4, 127]$	`labels[..., 21:]`
Loss	`mask`	$[4, 127]$	`attn_mask[..., 21:]`
Loss	`per_sample_loss`	$[4]$	Masked mean

Algorithm 1 Context-Aware Dual-Path Training

Require: Dataset $\mathcal{D}$, model M_θ, margin weight λ, epochs E

for epoch $= 1$ **to** E **do**
  for batch $\mathcal{B} \in \mathcal{D}$ **do**
    $\mathbf{x} \leftarrow \mathcal{B}$.`pixel_values` ▷ Shared image input

    // Forward Pass 1: Contextual Path
    $\hat{\mathbf{y}}_{\text{ctx}} \leftarrow M_\theta(\mathbf{x},\ \mathcal{B}$.`input_ids`, `context_length`$=C)$
    $\boldsymbol{\ell}^{\text{ctx}} \leftarrow \textsc{PerSampleLoss}(\hat{\mathbf{y}}_{\text{ctx}},\ \mathcal{B}$.`labels`$)$ ▷ Shape: $[B]$

    // Forward Pass 2: Isolation Path
    $\hat{\mathbf{y}}_{\text{iso}} \leftarrow M_\theta(\mathbf{x},\ \mathcal{B}$.`input_ids_iso`, `context_length`$=0)$
    $\boldsymbol{\ell}^{\text{iso}} \leftarrow \textsc{PerSampleLoss}(\hat{\mathbf{y}}_{\text{iso}},\ \mathcal{B}$.`labels_iso`$)$ ▷ Shape: $[B]$

    // Combined Loss with Margin
    $\mathbf{m} \leftarrow \lambda \cdot \max(\mathbf{0},\ \boldsymbol{\ell}^{\text{ctx}} - \boldsymbol{\ell}^{\text{iso}})$ ▷ Margin term, shape: $[B]$
    $\mathcal{L} \leftarrow \text{mean}(\boldsymbol{\ell}^{\text{ctx}} + \boldsymbol{\ell}^{\text{iso}} + \mathbf{m})$ ▷ Scalar

    $\theta \leftarrow \theta - \eta \cdot \nabla_\theta \mathcal{L}$ ▷ Gradient descent (with AMP)
  end for
  Evaluate on validation set (dual-path)
  Save checkpoint
end for

5.4 Dimension Trace — Isolation Path

For comparison, the isolation path (C=0):

Table 5.2: Dimension trace through the isolation forward pass.

Stage	Tensor	Shape	Notes
Input	`input_ids`	$[4, 128]$	T=128 (no context)
Reorder	`combined_embeddings`	$[4, 256, 768]$	I+T
LM Head	`logits`	$[4, 256, 1420]$	
Loss	`skip_positions`	128	0+128
Loss	`shift_logits`	$[4, 127, 1420]$	`logits[..., 128:-1, :]`
Loss	`shift_labels`	$[4, 127]$	`labels[..., 1:]`

Note that when C=0: `labels[..., 0+1:] = labels[..., 1:]`—identical to the original code, ensuring backward compatibility.

5.5 Validation and Checkpointing

The `evaluate_context_model()` function mirrors the training loop but without gradient computation. It reports:

- Combined loss: $\mathcal{L}_{\text{ctx}} + \mathcal{L}_{\text{iso}}$
- Individual losses: $\mathcal{L}_{\text{ctx}}$ and $\mathcal{L}_{\text{iso}}$ separately
- Average accuracy: $(\text{Acc}_{\text{ctx}} + \text{Acc}_{\text{iso}})/2$

Checkpoints are saved with the naming convention `checkpoint_context_epoch_{N}.pt` for context mode and `checkpoint_epoch_{N}.pt` for standard mode, with the corrected 9-argument signature matching `save_checkpoint()` in `utils.py`.

Chapter 6

Inference with Context

6.1 Sequential Context Propagation

At inference time, context comes from the *previously-recognized* word in the document. The inference proceeds sequentially across words within each line:

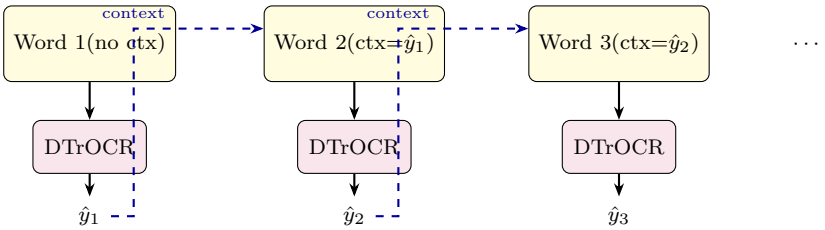

Figure 6.1: Inference pipeline with sequential context propagation. The recognized text of word i becomes the context for recognizing word i+1. The first word has no context.

6.2 Current State and Required Modifications

The current `generate()` method (model.py:271–340) does *not* pass `context_length` during generation—it defaults to 0, meaning inference currently operates in isolation mode only.

To enable context-aware inference, the following modifications are needed:

1. **Accept context text**: The `generate()` method should accept an optional `context_text` parameter.
2. **Process context**: Use the processor with `context_text` to create the contextual input sequence.
3. **Pass context_length**: Forward `context_length` to the model during the prefill step.
4. **KV cache handling**: Context tokens are only processed in the first forward pass (prefill); subsequent autoregressive steps use the KV cache and need no special handling.
5. **Orchestration**: The inference script (`extract_single_page.py`) must maintain state—the recognized text of each word—to provide as context for the next word.

Error propagation risk. If word i is misrecognized, the incorrect text becomes context for word i+1, potentially causing cascading errors. Mitigation strategies include confidence thresholding (only use context when the model is confident in the previous recognition) and beam search across word boundaries.

Chapter 7

Codebase Changes

This chapter provides side-by-side comparisons of all modified files, with original code in red-bordered boxes and new extension code in green-bordered boxes.

7.1 Summary of All Changes

Table 7.1: Summary of all file modifications.

File	Lines +/−	Original	Current	Nature of Change
`config.py`	+2 / −0	48 lines	50 lines	Added `max_context_length`
`data.py`	+2 / −0	28 lines	30 lines	Added output fields
`dataset.py`	+106 / −0	58 lines	164 lines	New `ContextAwareDataset`
`processor.py`	+93 / −6	58 lines	151 lines	Context tokenization pipeline
`model.py`	+82 / −28	556 lines	642 lines	Embedding reorder, loss fix
`train.py`	+203 / −49	82 lines	252 lines	Dual-path training overhaul

7.2 `config.py` — Added `max_context_length`

Original `config.py:22` (parameter list)

```
bn_vocab_file: str = '...',
):
```

New (Extension) `config.py:22-23` (parameter list)

```
bn_vocab_file: str = '...',
max_context_length: Optional[int] = 20,
):
```

Original `config.py:__init__` body

```
self.bn_vocab_file = bn_vocab_file
# (no max_context_length)
```

New (Extension) `config.py:__init__` body, line 34

```
self.max_context_length =
    max_context_length
self.bn_vocab_file = bn_vocab_file
```

7.3 data.py — New Output Fields

Original data.py:DTrOCRLMHeadModelOutput

```python
@dataclass
class DTrOCRLMHeadModelOutput:
    logits: torch.FloatTensor
    loss: Optional[...] = None
    accuracy: Optional[...] = None
    past_key_values: Optional[...] = None
```

New (Extension) data.py:DTrOCRLMHeadModelOutput

```python
@dataclass
class DTrOCRLMHeadModelOutput:
    logits: torch.FloatTensor
    loss: Optional[...] = None
    accuracy: Optional[...] = None
    past_key_values: Optional[...] = None
    per_sample_loss: Optional[...] = None  # NEW
```

Original data.py:DTrOCRProcessorOutput

```python
@dataclass
class DTrOCRProcessorOutput:
    pixel_values: Optional[...] = None
    input_ids: Optional[...] = None
    attention_mask: Optional[...] = None
    labels: Optional[...] = None
```

New (Extension) data.py:DTrOCRProcessorOutput

```python
@dataclass
class DTrOCRProcessorOutput:
    pixel_values: Optional[...] = None
    input_ids: Optional[...] = None
    attention_mask: Optional[...] = None
    labels: Optional[...] = None
    context_length: Optional[int] = None  # NEW
```

7.4 dataset.py — New ContextAwareDataset

The original `HandwrittenDataset` class (lines 11–43) is **unchanged**. The following is entirely **new code**:

New (Extension) dataset.py:46-139 -- ContextAwareDataset (NEW)

```python
class ContextAwareDataset(Dataset):
    """Dataset for context-aware HTR. Each sample includes
    the current word image and previous word's text as context."""

    def __init__(self, images_dir, json_dir, config, data_frame=None):
        super().__init__()
        self.images_dir = images_dir
        self.json_dir = json_dir
        self.config = config
        if data_frame is None:
            self.df = self._build_dataframe_from_json()
        else:
            self.df = data_frame
        self.processor = DTrOCRProcessor(config, add_eos_token=True, add_bos_token=True)

    def _build_dataframe_from_json(self):
        """Build dataframe from JSON files with previous word context."""
        data = []
        json_files = sorted([f for f in os.listdir(self.json_dir)
                             if f.endswith('.json')])
        for json_file in json_files:
            with open(os.path.join(self.json_dir, json_file), 'r') as f:
                json_data = json.load(f)
            data.append({
                'image_id': os.path.basename(json_data['output_path']),
                'text': json_data['text']
            })
        df = pd.DataFrame(data)
        df['prev_text'] = df['text'].shift(1).fillna("")  # Context
```

```python
        return df

    def __getitem__(self, index):
        row = self.df.iloc[index]
        image = Image.open(os.path.join(self.images_dir,
                            row['image_id'])).convert('RGB')
        text, prev_text = str(row['text']), str(row['prev_text'])

        # Path 1: WITH context
        inputs_ctx = self.processor(images=image, texts=text,
            context_text=prev_text, padding=True, return_labels=True)

        # Path 2: WITHOUT context (isolation)
        inputs_iso = self.processor(images=image, texts=text,
            context_text="", padding=True, return_labels=True)

        return {
            'pixel_values': inputs_ctx.pixel_values[0],
            'input_ids': inputs_ctx.input_ids[0],
            'attention_mask': inputs_ctx.attention_mask[0],
            'labels': inputs_ctx.labels[0],
            'context_length': inputs_ctx.context_length,
            'input_ids_isolated': inputs_iso.input_ids[0],
            'attention_mask_isolated': inputs_iso.attention_mask[0],
            'labels_isolated': inputs_iso.labels[0],
            'context_length_isolated': inputs_iso.context_length,
        }
```

New (Extension) dataset.py:152-163 -- split_context_data (NEW)

```python
def split_context_data(images_dir, json_dir, config,
                       test_size=0.05, random_seed=42):
    """Split context-aware dataset into train and test sets."""
    dataset = ContextAwareDataset(images_dir, json_dir, config)
    df = dataset.df
    train_df, test_df = train_test_split(df, test_size=test_size,
                                         random_state=random_seed)
    train_dataset = ContextAwareDataset(images_dir, json_dir, config,
                       data_frame=train_df.reset_index(drop=True))
    test_dataset = ContextAwareDataset(images_dir, json_dir, config,
                       data_frame=test_df.reset_index(drop=True))
    return train_dataset, test_dataset
```

7.5 processor.py — Context Tokenization Pipeline

Original processor.py:__init__ and __call__

```python
class DTrOCRProcessor:
  def __init__(self, config, ...):
    self.vit_processor = AutoImageProcessor
      .from_pretrained(...)
    self.tokeniser = BnGraphemizerProcessor(
      config.bn_vocab_file, ...)

  def __call__(self, images=None,
      texts=None, return_labels=False,
      ...):
    text_inputs = self.tokeniser(
      texts, padding=padding
    ) if texts is not None else None

    image_inputs = self.vit_processor(
      images, ...
    ) if images is not None else None

    return DTrOCRProcessorOutput(
      pixel_values=...,
      input_ids=...,
      attention_mask=...,
      labels=...
    )
```

New (Extension) processor.py:__init__ and __call__ (extended)

```python
class DTrOCRProcessor:
  def __init__(self, config, ...):
    self.max_context_length = getattr(
      config, 'max_context_length', 20)
        # NEW
    self.vit_processor = ...
    self.tokeniser = ...

  def __call__(self, images=None,
      texts=None,
      context_text=None,  # NEW PARAM
      return_labels=False, ...):
    context_length = 0
    text_inputs = self.tokeniser(texts,
        ...)

    # --- NEW: Context prepending ---
    if context_text is not None \
       and context_text != "" \
       and text_inputs is not None:
      ctx_ids = self.tokeniser(
        context_text, padding=False)
      ctx_ids = ctx_ids[:max_ctx_len - 1]
      ctx_with_sep = ctx_ids + [SEP]
      # Pad to fixed length
      pad_needed = max_ctx_len - len(ctx_with_sep)
      ctx_padded = ctx_with_sep + [PAD]*pad_needed
      ctx_mask = [1]*len(ctx_with_sep) \
                 + [0]*pad_needed
      context_length = max_ctx_len  # FIXED
      combined = ctx_padded + target_ids
      text_inputs['input_ids'] = combined
      text_inputs['attention_mask'] = ...

    return DTrOCRProcessorOutput(
      ...,
      context_length=context_length  # NEW
    )
```

7.6 model.py — DTrOCRModel.forward() — Embedding Reordering

Original model.py:51-89 (forward signature + embeddings)

```python
def forward(self, pixel_values, input_ids,
    position_ids=None, past_key_values=None,
    attention_mask=None, use_cache=False,
) -> DTrOCRModelOutput:
    # ...
    patch_emb = self.patch_embeddings(pixel_values) if past_length == 0 else None
    token_emb = self.token_embedding(input_ids)

    if patch_emb is not None:
      patch_and_token = torch.concat([patch_emb, token_emb], dim=-2)
    else:
      patch_and_token = token_emb

    # ... attention mask ...
    attention_mask = torch.concat([
      torch.ones(..., patch_emb.shape[-2], ...),
      attention_mask
    ], dim=-1)
```

New (Extension) model.py:43-143 (extended forward)

```python
def forward(self, pixel_values, input_ids,
    position_ids=None, past_key_values=None,
    attention_mask=None, use_cache=False,
    context_length: int = 0,        # NEW
) -> DTrOCRModelOutput:
    # ...
    patch_emb = self.patch_embeddings(...)
    token_emb = self.token_embedding(input_ids)

    # NEW: 3-way reorder
    if patch_emb is not None and context_length > 0:
      ctx_emb = token_emb[:, :context_length, :]
      tgt_emb = token_emb[:, context_length:, :]
      combined = torch.concat([
        ctx_emb, patch_emb, tgt_emb], dim=-2)
    elif patch_emb is not None:
      combined = torch.concat([patch_emb, token_emb], dim=-2)
    else:
      combined = token_emb

    # NEW: 3-way mask reorder
    if context_length > 0 and patch_emb is not None:
      ctx_mask = attn_mask[:, :context_length]
      tgt_mask = attn_mask[:, context_length:]
      patch_mask = torch.ones(..., 128, ...)
      attn_mask = torch.concat(
        [ctx_mask, patch_mask, tgt_mask], dim=-1)
    else:
      # Original behavior
      attn_mask = torch.concat([torch.ones(...), attn_mask], dim=-1)
```

7.7 model.py — DTrOCRLMHeadModel.forward() — Loss Computation

This is the most critical change, fixing Bugs 1 and 2:

Original model.py:148-176 (loss computation)

```python
def forward(self, pixel_values, input_ids,
    ..., labels=None,
) -> DTrOCRLMHeadModelOutput:
    transformer_output = self.transformer(
      pixel_values=pixel_values,
      input_ids=input_ids, ...)
    logits = self.language_model_head(...)

    loss, accuracy = None, None
    if labels is not None:
      shift_logits = logits[..., self.image_embedding_length:-1, :]
      shift_labels = labels[..., 1:]

      loss_fct = CrossEntropyLoss(reduction="none")
      loss = loss_fct(shift_logits.view(-1, ...),
                      shift_labels.view(-1))
      # ...
      if attention_mask is not None:
        mask = attention_mask[..., 1:].reshape(-1)
        loss = (mask * loss).sum() / mask.sum()
        accuracy = (mask * matches).sum() / mask.sum()
      else:
        loss = loss.mean()

    return DTrOCRLMHeadModelOutput(
      loss=loss, logits=logits,
      accuracy=accuracy, ...)
```

New (Extension) model.py:184-268 (fixed loss + per-sample)

```python
def forward(self, pixel_values, input_ids,
    ..., labels=None,
    context_length: int = 0, # NEW
    return_per_sample_loss: bool=False, # NEW
) -> DTrOCRLMHeadModelOutput:
    transformer_output = self.transformer(
      ..., context_length=context_length) # NEW
    logits = self.language_model_head(...)

    loss, accuracy, per_sample_loss = None,None,None
    if labels is not None:
      # FIX: skip context + image positions
      skip = context_length + self.image_emb_len
      shift_logits = logits[..., skip:-1, :]
      # FIX: skip context tokens in labels
      shift_labels = labels[..., context_length + 1:]

      loss_fct = CrossEntropyLoss(reduction="none")
      loss_per_token = loss_fct(...)

      # NEW: per-sample computation
      loss_2d = loss_per_token.view(B, seq_len)
      if attention_mask is not None:
        # FIX: skip context in mask
        mask = attention_mask[..., context_length + 1:].reshape(B, seq_len)
        per_sample_loss = (mask * loss_2d).sum(1) \
                          / mask.sum(1)
        loss = per_sample_loss.mean()

    return DTrOCRLMHeadModelOutput(
      ..., per_sample_loss=per_sample_loss
      if return_per_sample_loss else None)  # NEW
```

Key changes marked:

- **Bug 1 Fix (line 227):** `labels[..., 1:]` → `labels[..., context_length + 1:]`
- **Bug 2 Fix (line 248):** `attention_mask[..., 1:]` → `attention_mask[..., context_length + 1:]`
- **New:** `context_length` and `return_per_sample_loss` parameters
- **New:** Per-sample loss computation with 2D reshape

7.8 train.py — Complete Overhaul

Original train.py (original -- 82 lines, no context)

```python
import tqdm
from utils import ... , evaluate_model, ...
from dataset import split_data
# No argparse
# Hardcoded paths
images_dir = "../sample_train/images"
labels_file = "../sample_train/labels/label.csv"
config = DTrOCRConfig()
train_ds, val_ds = split_data(...)
# Hardcoded hyperparameters
train_dl = DataLoader(train_ds, batch_size=32, ...)
model = DTrOCRLMHeadModel(config)
model = torch.compile(model)  # Required
optimizer = torch.optim.Adam(..., lr=1e-4)
EPOCHS = 10

for epoch in range(EPOCHS):
  for inputs in tqdm.tqdm(train_dl, ...):
    optimizer.zero_grad()
    inputs = send_inputs_to_device(inputs, ...)
    with torch.autocast(...):
      outputs = model(**inputs)
    scaler.scale(outputs.loss).backward()
    scaler.step(optimizer)
    scaler.update()
    epoch_losses.append(outputs.loss.item())

  val_loss, val_acc = evaluate_model(...)
  print(f"Epoch {epoch+1} ...")
  # No checkpoint saving
  # No final model saving
```

New (Extension) train.py (new -- 252 lines, dual-path)

```python
import tqdm, argparse            # NEW
from utils import ..., save_checkpoint, save_final_model
from dataset import split_data, split_context_data

# NEW: argparse with 7 parameters
parser = argparse.ArgumentParser(...)
parser.add_argument('--context', ...)
parser.add_argument('--margin_lambda', default=0.5)
parser.add_argument('--batch_size', '--epochs', '--lr')
args = parser.parse_args()

# NEW: mode selection
if args.context:
  train_ds, val_ds = split_context_data(...)
else:
  train_ds, val_ds = split_data(...)

# Bug 3 fix: function BEFORE loop
def evaluate_context_model(model, dl, device):
  """Dual-path validation."""  # 40 lines
  ...

for epoch in range(EPOCHS):
  for inputs in tqdm.tqdm(train_dl, ...):
    if args.context:
      # NEW: dual-path forward
      out_ctx = model(..., context_length=C,
                      return_per_sample_loss=True)
      out_iso = model(..., context_length=0,
                      return_per_sample_loss=True)
      # Bug 4 fix: margin loss
      base = per_sample_ctx + per_sample_iso
      margin = lambda * clamp(ctx - iso, min=0)
      loss = (base + margin).mean()
    else:
      # Standard (unchanged)
      outputs = model(**inputs)
      loss = outputs.loss

  # Bug 6 fix: correct save_checkpoint args
  save_checkpoint(model, optimizer, epoch+1,
    train_loss, val_loss, train_acc, val_acc,
    './', f'checkpoint_...epoch_{epoch+1}.pt')
save_final_model(model, 'final_...model.pth')
```

Chapter 8

Bug Discovery and Resolution

The context-aware extension initially contained 6 bugs that prevented training from completing a single epoch. These were discovered through systematic code auditing and resolved in commits `d162227` and `3b72a7d`.

8.1 Bug Summary

Table 8.1: All 6 bugs discovered, in crash order. Red rows crash; yellow is a silent logic error.

#	File:Line	Type	Crash?	Order	Fix Applied
5	`processor.py:79`	Variable-length context	Yes	1st	Fixed-length padding to C=20
1	`model.py:227`	Label dim mismatch	Yes	2nd	`labels[..., ctx_len+1:]`
2	`model.py:248`	Mask reshape mismatch	Yes	3rd	`attn_mask[..., ctx_len+1:]`
3	`train.py:176`	Func before definition	Yes	4th	Moved function above loop
6	`train.py:194`	Wrong save_checkpoint args	Yes	5th	Fixed to 9 arguments
4	`train.py:124`	Wrong loss formula	No	Always	Replaced with margin term

8.2 Bug 1 — Label Dimension Mismatch

Location: `model.py`, line 225 (original), `DTrOCRLMHeadModel.forward()`

Problem. The original code shifts labels with `labels[..., 1:]` producing shape $[B, C{+}T{-}1]$, but `shift_logits` (after skipping context + image positions) has shape $[B, T{-}1, V]$. When $C > 0$, $C{+}T{-}1 \neq T{-}1$, causing `CrossEntropyLoss` to fail with a size mismatch error.

Numerical trace (C=3, T=6, I=128):

$$\texttt{shift_logits} = \texttt{logits[..., 131:-1, :]} \quad \text{shape: } [B, 5, V] \quad (T{-}1 = 5)$$
$$\texttt{shift_labels (BUG)} = \texttt{labels[..., 1:]} \quad \text{shape: } [B, 8] \quad (C{+}T{-}1 = 8)$$
$$\texttt{shift_labels (FIX)} = \texttt{labels[..., 4:]} \quad \text{shape: } [B, 5] \quad (T{-}1 = 5)\ \checkmark$$

Backward compatibility. When C=0: `labels[..., 0+1:] = labels[..., 1:]`—identical to original.

8.3 Bug 2 — Attention Mask Reshape Mismatch

Location: `model.py`, line 246 (original)

Problem. The `attention_mask` variable in `DTrOCRLMHeadModel.forward()` is the *original, un-reordered* mask from the caller (shape $[B, C{+}T]$), not the reordered mask inside `DTrOCRModel.forward()`. Python's parameter passing means rebinding the local variable inside the inner function does not affect the outer reference. The reshape `.reshape(B, T-1)` fails because $C{+}T{-}1 \neq T{-}1$.

Fix. Same pattern as Bug 1: `attention_mask[..., context_length + 1:]`.

8.4 Bug 3 — Function Called Before Definition

Location: `train.py`, line 176 (call) vs. line 202 (definition)

Problem. Python executes module-level code top-to-bottom. The training loop at line 69 begins before the interpreter reaches the `evaluate_context_model` definition at line 202. When the first epoch ends and validation is invoked at line 176, a `NameError` results.

Fix. Move the function definition to before the training loop (now at line 63).

8.5 Bug 4 — Wrong Loss Formula

Location: `train.py`, lines 124–135 (original)

Problem. The original code implemented *difficulty weighting* (upweighting hard samples based on isolation loss) instead of the intended *margin penalty* (penalizing samples where context hurts). See **??** for the mathematical comparison.

Fix. Replace with the margin term:

```
margin_term = args.margin_lambda * torch.clamp(
    per_sample_loss_ctx - per_sample_loss_iso, min=0.0)
weighted_loss = (combined_base_loss + margin_term).mean()
```

8.6 Bug 5 — Variable-Length Context Breaking DataLoader

Location: `processor.py`, lines 79–125 (original)

Severity: **Critical**—crashes immediately when DataLoader forms the first batch.

Problem. Context was tokenized without padding, producing variable-length `input_ids` tensors. When the DataLoader calls `torch.stack()`, tensors of different lengths cause a `RuntimeError`.

Additionally: Even within a batch, `context_length` varied per sample. The training loop took only `inputs['context_length'][0].item()`—the first sample's value—silently applying the wrong offset for all other samples.

Fix. Pad all contexts to fixed `max_context_length = 20` with truncation, SEP token, and padding (see **??**).

8.7 Bug 6 — `save_checkpoint` Argument Mismatch

Location: `train.py`, line 194 (original)

Problem. The function `save_checkpoint()` in `utils.py` expects 9 positional arguments: `model, optimizer, epoch, train_loss, val_loss, train_acc, val_acc, checkpoint_dir, checkpoint_name`. The original call passed only 6 arguments, causing a `TypeError`.

Fix. Add the missing `train_accuracies[-1]`, `validation_accuracies[-1]`, and `'./'` arguments.

Chapter 9

Test Infrastructure

9.1 Test Suite Overview

The extension includes a comprehensive test suite of 11 files:

Table 9.1: Test files in the `test_scripts/` directory.

File	Description
`00_run_all_tests.sh`	Master orchestrator (`-quick` or `-full` mode)
`01_test_dataset_loading.py`	Standard and context dataset loading validation
`02_test_model_forward.py`	Forward pass dimensions, dual-path, difficulty weights
`03_quick_train_standard.sh`	Quick standard training run (1 epoch)
`04_quick_train_context.sh`	Quick context training run (1 epoch)
`05_train_context_full.sh`	Full context training
`06_compare_context_vs_standard.sh`	Side-by-side comparison
`07_test_all_batch_sizes.sh`	Batch size sweep
`08_test_learning_rates.sh`	Learning rate sweep
`09_validate_context_benefit.py`	Statistical validation that context helps
`10_diagnostic_context_suite.py`	Comprehensive diagnostic: 18 test cases, 1020 lines

9.2 Key Diagnostic Tests

The comprehensive diagnostic suite (`10_diagnostic_context_suite.py`) covers:

1. **Tests 1–2**: Isolation and context forward pass dimension traces
2. **Test 3**: Shift alignment sweep across $C \in \{0, 1, 2, \ldots, 25\}$—verifies `shift_logits` and `shift_labels` match for every context length
3. **Test 4**: Label content correctness (the actual token IDs are correct, not just shapes)
4. **Test 5**: Attention mask alignment (mask has correct length after context skip)
5. **Tests 6–7**: Margin term activation logic and full loss formula numerical verification
6. **Test 8**: Full dual-path forward pass simulation
7. **Test 9**: Gradient flow verification (gradients are non-zero and finite)
8. **Test 10**: Backward compatibility with `context_length=0`
9. **Tests 11–12**: Edge cases (batch_size=1, heavy padding at 90%)

10. **Test 13**: Processor fixed-length context padding verification
11. **Test 14**: DataLoader stacking simulation (the original Bug 5 crash scenario)
12. **Test 15**: `save_checkpoint` signature inspection
13. **Test 16**: `evaluate_context_model` definition order check
14. **Test 17**: Position embedding budget validation ($C + I + T \leq$ `max_position_embeddings`)
15. **Test 18**: Margin term with actual model outputs (end-to-end)

Chapter 10

Discussion and Future Work

10.1 Architectural Trade-offs

Shared weights. Both the contextual and isolation paths share the same transformer weights. This is computationally expensive (2 forward passes per sample) but ensures the model learns a unified representation that works with or without context.

Fixed context length. Padding all contexts to C=20 wastes some positions on short context words. Dynamic batching (grouping similar-length contexts) could improve efficiency but adds complexity.

Position embedding extension. Increasing `max_position_embeddings` from 256 to 320 means positions 256–319 are randomly initialized. When fine-tuning a pretrained model, these new positions start without prior knowledge—early training may be unstable for samples that use the extended range.

10.2 Limitations

1. **Error propagation**: At inference, a misrecognized word provides incorrect context for the next word, potentially causing cascading errors.
2. **Single-word context**: The current design uses only the immediately preceding word. A sliding window over multiple previous words could capture richer linguistic context.
3. **Inference not yet implemented**: The `generate()` method does not currently support context-aware decoding. This is a training-only extension.
4. **No benchmark results yet**: The context-aware model has not been evaluated on BN-HTRd or Bongabdo benchmarks.